Command Line & HPC Cluster Tutorial for New Lab Members

From Zero to Running Your First Cluster Job

Lab	COMFHA Computational Lab
Department	Kasetsart University
Audience	New lab members with no prior terminal experience
Version	1.0 (2026)

This tutorial guides you step by step from opening a terminal for the first time to submitting your own job on the KU Cluster and Nontri HPC. No prior programming or Linux experience is required. Complete each section in order and attempt every exercise before moving on.

Contents

Part I The Command Line

1. What Is a Terminal?

When you open a program like a file manager or a web browser, you interact with it through icons and menus. A **terminal** (also called a *shell*, *command line*, or *command prompt*) is a text-based way to interact with your computer. Instead of clicking, you type instructions and the computer responds.

Why scientists use the terminal

- Run programs that have no graphical interface (e.g. GROMACS, Python scripts)
- Automate repetitive tasks with a single command
- Connect to remote supercomputers (clusters)
- Work faster once you know a few commands

1.1. The Shell

The program that reads your commands and runs them is called a **shell**. The most common shell is **Bash** (Bourne Again SHell), which is the default on most Linux systems and clusters. macOS uses **Zsh** (Z Shell) by default, which is nearly identical for everyday use.

1.2. Opening a Terminal

Operating System	How to open a terminal
macOS	`Cmd + Space` → type `Terminal` → press `Enter`
Ubuntu/Debian	`Ctrl + Alt + T`
Windows	Install *Git Bash* or use *Windows Subsystem for Linux (WSL)*

When the terminal opens you will see something like this:

```
yourusername@computername ~ %
```

This is the **prompt**. It tells you who you are, which machine you are on, and where you are in the file system. The `%` (or `$`) is where you type your command.

△ Important

Never close the terminal mid-task by just clicking the X. Type `exit` or press `Ctrl+D` to close it properly.

2. Your First Commands

Every command follows the same basic structure:

```
command  [options]  [arguments]
```

- **command** — the program you want to run
- **options** — flags that change behavior, usually start with `-`
- **arguments** — the target (file, directory, etc.)

2.1. whoami, date, echo

```
whoami            # prints your username
date              # prints today's date and time
echo "Hello!"     # prints whatever you write
```

```
$ whoami
pao
$ date
Mon Jun 23 10:05:42 +07 2026
$ echo "Hello, lab!"
Hello, lab!
```

Tip

Text after a `#` in a command is a **comment** — the shell ignores it. Comments are used in this tutorial to explain what a line does.

Open your terminal and run all three commands above. What is your username? What is today's date?

3. Navigating the File System

The file system is organized as a tree of folders (called **directories**). On Linux and macOS the root of the tree is `/`. Your personal folder is called the **home directory**

and is written as ~ (a tilde).

```
/
+-- home/
|   \-- pao/          <- your home directory (~)
|       +-- Documents/
|       +-- Workspace/
|       \-- scripts/
+-- etc/
\-- usr/
```

3.1. pwd — Print Working Directory

`pwd` tells you where you currently are.

```
pwd
```

```
/Users/pao
```

3.2. ls — List Files

```
ls              # list files in current directory
ls -l           # long format (shows permissions, size,
   date)
ls -lh          # same but sizes in human-readable form (KB
   , MB)
ls -la          # include hidden files (names starting with
    .)
ls Documents/   # list a specific directory
```

```
$ ls -lh
total 48K
drwxr-xr-x  3 pao  staff   96B Jun 23 09:00 Documents
drwxr-xr-x  5 pao  staff  160B Jun 22 14:30 Workspace
-rw-r--r--  1 pao  staff  2.4K Jun 20 11:00 notes.txt
```

3.3. cd — Change Directory

```
cd Documents        # go into the Documents folder
cd ..               # go up one level (parent directory)
cd ../..            # go up two levels
```

```
cd ~                    # go to your home directory (always
   works)
cd -                    # go back to the previous directory
```

Tip

Tab completion is your best friend. Start typing a folder or file name and press `Tab` — the shell will complete it for you. Press `Tab` twice to see all possible matches.

1. Run `pwd` to see where you are.
2. Run `ls -lh` to list your files.
3. Navigate into one of the folders shown, then use `cd ..` to come back.
4. Try typing `cd Doc` then press `Tab`. What happens?

4. Working with Files and Directories

4.1. mkdir — Make Directory

```
mkdir my_project              # create a folder
mkdir -p data/raw/outputs     # create nested folders at
   once
```

4.2. touch — Create an Empty File

```
touch notes.txt               # create an empty file called
   notes.txt
touch run.sh analysis.py      # create multiple files at
   once
```

4.3. cp — Copy

```
cp notes.txt notes_backup.txt          # copy a file
cp -r my_project/ my_project_old/      # copy a directory (-r
    = recursive)
```

4.4. mv — Move or Rename

```
mv notes.txt Documents/                # move file to
   Documents/
```

```
mv notes.txt summary.txt          # rename a file (same
    directory)
mv my_project/ Workspace/         # move a folder
```

4.5. rm — Remove (Delete)

△ Important

There is no Recycle Bin on the command line. Once you run `rm`, the file is gone permanently. Be very careful, especially with `rm -r`.

```
rm notes.txt                  # delete a file
rm -r old_data/               # delete a directory and
    everything inside it
rm -i file.txt                # ask for confirmation before
    deleting
```

4.6. Quick Reference

Command	What it does
`mkdir` *name*	Create a directory
`touch` *file*	Create an empty file
`cp` *a b*	Copy *a* to *b*
`mv` *a b*	Move or rename *a* to *b*
`rm` *file*	Delete a file
`rm -r` *dir*	Delete a directory

1. Create a directory called `lab_practice`.
2. Inside it, create a file called `hello.txt`.
3. Copy `hello.txt` to `hello_copy.txt`.
4. Rename `hello_copy.txt` to `greeting.txt`.
5. List the directory to confirm both files exist.
6. Delete `greeting.txt`.

5. Viewing File Contents

5.1. cat

`cat` prints the entire file to the screen. Best for short files.

```
cat notes.txt
```

5.2. less

`less` opens a file for scrolling — press `Space` to go forward, `b` to go back, `q` to quit. Best for long files.

```
less trajectory_log.txt
```

5.3. head and tail

```
head notes.txt          # show first 10 lines
head -n 20 notes.txt    # show first 20 lines
tail notes.txt          # show last 10 lines
tail -f output.log      # keep watching a file as it grows (
   useful for job logs)
```

5.4. grep — Search Inside Files

```
grep "error" output.log          # find lines containing "
   error"
grep -i "warning" output.log     # case-insensitive search
grep -n "Step" md.log            # show line numbers
```

Tip

`tail -f` is extremely useful when you have a simulation running — it lets you watch the progress in real time without opening a new connection.

6. Editing Files with nano

nano is a simple text editor that runs inside the terminal.

```
nano notes.txt          # open (or create) notes.txt for
   editing
```

Once inside nano, you can type normally. The keyboard shortcuts shown at the bottom of the screen use ^ to mean `Ctrl`:

Shortcut	Action
`Ctrl+O` then `Enter`	Save (Write Out)
`Ctrl+X`	Exit nano
`Ctrl+K`	Cut the current line
`Ctrl+U`	Paste
`Ctrl+W`	Search
`Ctrl+\`	Find and Replace

1. Open `nano hello.txt` inside your `lab_practice` folder.
2. Type: `My name is [your name] and I am learning the terminal.`
3. Save with `Ctrl+O` then `Enter`.
4. Exit with `Ctrl+X`.
5. Verify with `cat hello.txt`.

7. Productivity Tips

7.1. Command History

Press the ↑ arrow key to cycle through previous commands. You do not need to retype them.

```
history              # show all recent commands
history | grep cd    # find all times you used cd
```

7.2. Ctrl Shortcuts

Shortcut	What it does
`Ctrl+C`	Cancel the currently running command
`Ctrl+Z`	Pause the running command (resume with `fg`)
`Ctrl+L`	Clear the screen
`Ctrl+A`	Jump to beginning of the line
`Ctrl+E`	Jump to end of the line
`Ctrl+D`	Log out / close the terminal

7.3. The Pipe Operator |

The pipe | sends the output of one command as input to another. It is one of the most powerful features of the shell.

```
ls -lh | grep ".txt"     # list only .txt files
cat data.log | tail -20 | grep "STEP"  # last 20 lines
    containing "STEP"
```

7.4. Wildcards

```
ls *.txt           # list all files ending in .txt
rm output_*.log    # delete all files matching the pattern
cp *.py scripts/   # copy all Python files into scripts/
```

Part II SSH — Connecting to Remote Computers

8. What Is SSH?

SSH (Secure Shell)

SSH is a protocol that lets you log in to a remote computer over the network and use its terminal as if you were sitting in front of it. All communication is encrypted — no one can intercept your commands or data.

In our lab you will use SSH to connect to two systems:

1. **KU Cluster** (`ku-cluster`) — general-purpose cluster at Kasetsart University. Used for MD simulations with GROMACS.
2. **Nontri HPC** (`ku-ai`) — GPU cluster (Nontri) at Kasetsart University. Used for GPU-accelerated jobs.

8.1. Basic SSH Syntax

```
ssh username@hostname
```

For example:

```
ssh pao@158.108.25.40       # connect to KU Cluster
ssh pao@br1.paas.ku.ac.th   # connect to Nontri HPC
```

The first time you connect, SSH will ask you to verify the server's fingerprint. Type `yes` and press `Enter`.

9. SSH Keys — Password-Free Login

Typing a password every time is tedious and less secure. **SSH keys** are a pair of files:

- **Private key** — stays on *your* computer, never share it.
- **Public key** — placed on the remote server. Anyone with the matching private key can log in.

9.1. Step 1 — Generate a Key Pair

Run this on your **local machine** (your laptop or desktop):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

```
Generating public/private ed25519 key pair.
Enter file in which to save the key
   (/Users/pao/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in
   /Users/pao/.ssh/id_ed25519
Your public key has been saved in
   /Users/pao/.ssh/id_ed25519.pub
```

- Press `Enter` to accept the default file location.
- You may set a passphrase for extra security, or leave it empty.

This creates two files in `~/.ssh/`:

File	Description
`id_ed25519`	Your **private** key — **never share this**
`id_ed25519.pub`	Your **public** key — this goes to the server

9.2. Step 2 — Copy Your Public Key to the Server

```
# Easiest method (if ssh-copy-id is available):
ssh-copy-id -i ~/.ssh/id_ed25519.pub username@hostname

# Manual method:
cat ~/.ssh/id_ed25519.pub
# Copy the output, then on the remote server run:
# echo "PASTE_KEY_HERE" >> ~/.ssh/authorized_keys
```

After this you can log in without a password:

```
ssh username@158.108.25.40    # no password prompt!
```

△ Important

If you generate a *separate* key pair for each cluster, give them different names during `ssh-keygen` (e.g. `id_ed25519_ku`). Use the `-i` flag with SSH to specify which key to use:

```
ssh -i ~/.ssh/id_ed25519_ku username@158.108.25.40
```

9.3. Step 3 — Create an SSH Config File (Recommended)

Instead of typing long commands, you can create a shortcut file at `~/.ssh/config`:

```
nano ~/.ssh/config
```

Add the following (replace `your_username` with your actual username):

```
# KU Cluster
Host ku-cluster
    HostName 158.108.25.40
    User your_username
    IdentityFile ~/.ssh/id_ed25519_ku

# Nontri HPC
Host ku-ai
    HostName br1.paas.ku.ac.th
    User your_username
    IdentityFile ~/.ssh/id_ed25519
```

Save and exit nano (Ctrl+O, Enter, Ctrl+X).

Now you can connect with just:

```
ssh ku-cluster      # instead of ssh -i ... username@158
   .108.25.40
ssh ku-ai           # instead of ssh -i ... username@br1.
   paas.ku.ac.th
```

10. Connecting to the KU Cluster

10.1. Overview

Property	Details
Hostname	`158.108.25.40`
Short name	`ku-cluster` (after SSH config is set up)
Scheduler	SLURM
Recommended key	`id_ed25519_ku`

10.2. Logging In

```
ssh ku-cluster
```

You will see a welcome message followed by the cluster prompt:

```
Last login: Mon Jun 23 09:30:00 2026 from 10.31.74.100

[your_username@ku-cluster ~]$
```

10.3. First Things to Do After Login

```
pwd                    # confirm your home directory path
ls                     # see what is already there
df -h ~                # check how much disk space you have
```

10.4. Loading GROMACS

Software on clusters is managed with the `module` system. You load the software you need before running it:

```
module avail                    # list all available
   software
module avail gromacs            # search for GROMACS
   specifically
module load gromacs/2020.4      # load a specific version
gmx --version                   # confirm it loaded
```

Tip

To avoid loading modules manually every time, add the `module load` line to your `~/.bashrc` file on the cluster.

10.5. Logging Out

```
exit
```

11. Connecting to Nontri HPC (ku-ai)

11.1. Overview

Property	Details
Hostname	`br1.paas.ku.ac.th`
Short name	`ku-ai` (after SSH config is set up)
Scheduler	SLURM
GPU queue	`gpuq`
GROMACS	2022.6, 2024.1

11.2. Logging In

```
ssh ku-ai
```

```
[your_username@nontri ~]$
```

11.3. Loading GROMACS on Nontri

```
module avail gromacs
module load gromacs/2024.1     # GPU-enabled version
gmx --version
```

△ Important

Nontri uses GPU nodes for GROMACS. Always request a `gpuq` partition in your SLURM script (see Section 12.2). Running a GPU-optimized binary on CPU-only nodes will either fail or run very slowly.

12. Transferring Files Between Your Computer and a Cluster

12.1. scp — Secure Copy

`scp` works like `cp` but over SSH.

```
# Upload: local -> cluster
scp my_script.py ku-cluster:~/Workspace/
```

```
# Download: cluster -> local
scp ku-cluster:~/outputs/result.xvg ./

# Copy a whole directory (recursive)
scp -r local_folder/ ku-cluster:~/Workspace/

# Same for Nontri
scp analysis.py ku-ai:~/project/
```

12.2. rsync — Smart Sync (Recommended)

`rsync` only transfers files that have changed, making it much faster for large directories.

```
# Sync a local folder to the cluster
rsync -avz --progress local_folder/ ku-cluster:~/Workspace/
   local_folder/

# Download results back to your machine
rsync -avz ku-cluster:~/outputs/ ./outputs/
```

Flag	Meaning
`-a`	Archive mode (preserves timestamps, permissions, etc.)
`-v`	Verbose (show what is being transferred)
`-z`	Compress data during transfer (faster on slow networks)
`-progress`	Show transfer progress

Tip

Always use `rsync` instead of `scp` when syncing simulation output folders — trajectory files are large and rsync avoids re-sending what already arrived.

Part III Running Jobs on the Cluster (SLURM)

13. What Is a Job Scheduler?

A cluster is a shared resource — many researchers use it at the same time. A **scheduler** (our clusters use **SLURM**) manages the queue of jobs and decides which job runs on which node and when.

You submit a **job script** — a text file that describes what resources you need and what commands to run. SLURM puts your job in the queue and runs it when resources are available.

Key concepts

Node One physical computer in the cluster.
Core / CPU A single processing unit on a node.
Partition A group of nodes (e.g. GPU nodes, high-memory nodes).
Job Your submitted work unit.
Queue The list of waiting jobs.
Walltime The maximum time your job is allowed to run.

14. Checking the Cluster Status

```
sinfo                          # see all partitions and node
   status
squeue                         # see all jobs currently in
   the queue
squeue -u your_username        # see only YOUR jobs
squeue --start                 # see estimated start times
```

```
$ squeue -u pao
JOBID   PARTITION   NAME        USER   ST   TIME         NODES
   NODELIST
12345   main        run_nvt     pao    R    0:05:23      1
   node04
12346   gpuq        run_gpu     pao    PD   0:00:00      1
   (Priority)
```

Common **ST** (state) values:

State	Meaning
`R`	Running
`PD`	Pending (waiting in queue)
`CG`	Completing
`F`	Failed

15. Writing a SLURM Job Script

A job script is a regular shell script with special comments at the top that start with `#SBATCH`. These tell SLURM what resources you need.

15.1. Template for KU Cluster (CPU job)

```
#!/bin/bash
#SBATCH --job-name=my_simulation      # name shown in squeue
#SBATCH --output=slurm_%j.out         # stdout log (%j = job
    ID)
#SBATCH --error=slurm_%j.err          # stderr log
#SBATCH --nodes=1                     # number of nodes
#SBATCH --ntasks=1                    # number of MPI tasks
#SBATCH --cpus-per-task=6             # CPU cores per task
#SBATCH --mem=8G                      # total memory
#SBATCH --time=24:00:00               # max walltime (HH:MM:
   SS)
#SBATCH --partition=main              # partition name

# -- Environment setup
   ---------------------------------------
module load gromacs/2020.4

# -- Change to the directory you submitted from
   -------------
cd $SLURM_SUBMIT_DIR

# -- Your commands
   -----------------------------------------
echo "Job started on $(hostname) at $(date)"

gmx mdrun -v -deffnm run -ntmpi 1 -ntomp 6

echo "Job finished at $(date)"
```

Save this file as, for example, `submit.sh`.

15.2. Template for Nontri HPC (GPU job)

```
#!/bin/bash
#SBATCH --job-name=gpu_simulation
#SBATCH --output=slurm_%j.out
#SBATCH --error=slurm_%j.err
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --gres=gpu:1                    # request 1 GPU
#SBATCH --mem=16G
#SBATCH --time=48:00:00
#SBATCH --partition=gpuq                # GPU partition on
   Nontri

module load gromacs/2024.1

cd $SLURM_SUBMIT_DIR

echo "Job started on $(hostname) at $(date)"

# GPU-enabled GROMACS run
gmx mdrun -v -deffnm run -ntmpi 1 -ntomp 4 -gpu_id 0

echo "Job finished at $(date)"
```

15.3. Common #SBATCH Options

Option	Description
`-job-name=`*name*	Job name (appears in squeue)
`-output=`*file*	Standard output log file
`-error=`*file*	Error log file
`-nodes=`*N*	Number of nodes
`-ntasks=`*N*	Number of parallel tasks (MPI)
`-cpus-per-task=`*N*	CPU cores per task (OpenMP)
`-mem=`*size*	Total memory (e.g. 8G)
`-time=`*HH:MM:SS*	Maximum runtime
`-partition=`*name*	Which partition to use
`-gres=gpu:`*N*	Number of GPUs to request

16. Submitting and Managing Jobs

```
# Submit your job script
sbatch submit.sh

# Check the status of your jobs
squeue -u your_username

# Cancel a job (use the JOBID from squeue)
scancel 12345

# Cancel ALL your jobs
scancel -u your_username

# See details and resource usage of a completed job
sacct -j 12345 --format=JobID,JobName,State,Elapsed,MaxRSS

# See estimated start time
squeue --start -u your_username
```

```
$ sbatch submit.sh
Submitted batch job 12345

$ squeue -u pao
JOBID   PARTITION   NAME           USER   ST   TIME       NODES
12345   main        my_simulation  pao    R    0:01:45    1
```

16.1. Monitoring Your Running Job

Your job writes output to the log files you specified in the script. Watch them in real time with:

```
tail -f slurm_12345.out       # replace 12345 with your
    actual job ID
```

Press `Ctrl+C` to stop watching the file.

1. Log in to one of the clusters.
2. Create a file called `test_job.sh` using nano.
3. Paste a simple job script that runs `echo "Hello from the cluster!"`.
4. Submit it with `sbatch test_job.sh`.
5. Watch the queue with `squeue -u your_username`.
6. When the job finishes, read the output file with `cat`.

17. Keeping Jobs Running After You Log Out

Jobs submitted with `sbatch` continue running even after you disconnect — this is the whole point of a scheduler. However, if you run a program *directly* in the terminal (not via sbatch) and you disconnect, it will be killed.

Use screen or tmux for interactive sessions

If you need an interactive session that survives disconnection, use `screen` or `tmux`:

```
screen -S mysession      # start a named screen session
# ... run your commands ...
# Detach: Ctrl+A then D
screen -r mysession      # reattach after reconnecting
```

However, for all production jobs in our lab, always use `sbatch`.

Appendix Quick Reference Card

A.1 — Essential Commands

Command	What it does
`pwd`	Print current directory
`ls -lh`	List files (human-readable sizes)
`cd dir`	Change to directory
`cd ..`	Go up one level
`mkdir name`	Create directory
`touch file`	Create empty file
`cp a b`	Copy a to b
`mv a b`	Move/rename a to b
`rm file`	Delete file
`cat file`	Print file contents
`less file`	Scroll through file (q to quit)
`tail -f file`	Watch file in real time
`grep pat file`	Search for pattern in file
`nano file`	Open text editor
`history`	Show command history

A.2 — SSH & File Transfer

Command	What it does
`ssh ku-cluster`	Connect to KU Cluster
`ssh ku-ai`	Connect to Nontri HPC
`ssh-keygen -t ed25519`	Generate SSH key pair
`ssh-copy-id user@host`	Install public key on server
`scp file.txt ku-cluster:~/`	Upload file to cluster
`scp ku-cluster:~/file.txt .`	Download file from cluster
`rsync -avz dir/ ku-cluster:~/dir/`	Sync directory to cluster
`exit`	Log out

A.3 — SLURM Job Management

Command	What it does
`sbatch submit.sh`	Submit a job script
`squeue -u` *`username`*	Check your jobs
`squeue -start`	See estimated start times
`scancel` *`JOBID`*	Cancel a specific job
`scancel -u` *`username`*	Cancel all your jobs
`sinfo`	Show partition/node status
`sacct -j` *`JOBID`*	Show finished job details
`module avail`	List available software
`module load` *`name/version`*	Load software
`module list`	List currently loaded modules

A.4 — Cluster Connection Details

Cluster	SSH alias	Hostname	Scheduler
KU Cluster	`ku-cluster`	`158.108.25.40`	SLURM
Nontri HPC	`ku-ai`	`br1.paas.ku.ac.th`	SLURM (`gpuq`)

Getting Help

- **Manual pages:** type `man command` (e.g. `man ls`) for full documentation
- **Quick help:** type `command -help` for a short summary
- **SLURM docs:** `https://slurm.schedmd.com/documentation.html`
- **GROMACS docs:** `https://manual.gromacs.org`
- **Ask your supervisor or senior lab members** — we are here to help!